

waph-nagpalvv

WAPH-Web Application Programming and Hacking

Lab 1 Report

Instructor: Dr. Phu Phung

Student

Name: Viplav Nagpal

Email: nagpalvv@mail.uc.edu (viplavnagpal1704@gmail.com)

Short-bio: I love programming and building stuff. I took this course to learn how to build stuff and add security for the stuff I build, including websites, machine learning models and so much more.



Figure 1: Viplav's headshot

Lab 1 - Foundations of the Web

The lab's overview:

In this lab, we explore foundational web protocols by using Wireshark, Telnet, and curl to inspect and manually construct HTTP GET and POST requests. Additionally, we develop and deploy basic server-side web applications, including

a CGI program in C and an interactive user-input script in PHP. **Link to this lab report:** [<https://github.com/nagpalvv/waph-nagpalvv/tree/main/labs/lab1>]

Part 1: The Web and HTTP Protocol

Task 1: Familiar with the Wireshark tool and HTTP protocol

To examine the HTTP protocol, we started a Wireshark packet capture on our network interface and visited an unencrypted HTTP website. After stopping the capture, we applied the http filter to isolate the relevant traffic. This allowed us to inspect the raw data of the initial HTTP GET request sent by the browser and the corresponding 200 OK HTTP Response from the server. Finally, we used the “Follow > HTTP Stream” feature to view the entire plain-text conversation between the client and the server. Issues faced: Always got 404 Not Found as the HTTP response for <http://example.com/index.html> but worked completely fine for <http://example.com>

+	348	9.706546741	10.0.2.15	172.66.147.243	HTTP	395 GET / HTTP/1.1
+	352	9.730786130	172.66.147.243	10.0.2.15	HTTP	743 HTTP/1.1 200 OK (text/html)
+	375	9.816803942	10.0.2.15	172.66.147.243	HTTP	409 GET /favicon.ico HTTP/1.1
+	388	9.837714180	172.66.147.243	10.0.2.15	HTTP	686 HTTP/1.1 404 Not Found (text/html)

Frame 348: 395 bytes on wire (3160 bits), 395 bytes captured (3160 bits) on interface any, id 0	
Linux cooked capture v1	
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 172.66.147.243	
Transmission Control Protocol, Src Port: 50674, Dst Port: 80, Seq: 1, Ack: 1, Len: 339	
Hypertext Transfer Protocol	
GET / HTTP/1.1\r\n	
Host: example.com\r\n	
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:151.0) Gecko/20100101 Firefox/151.0\r\n	
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n	
Accept-Language: en-US,en;q=0.9\r\n	
Accept-Encoding: gzip, deflate\r\n	
Connection: keep-alive\r\n	
Upgrade-Insecure-Requests: 1\r\n	
Priority: u=0, i\r\n	
\r\n	
[Full request URI: http://example.com/]	
[HTTP request 1/2]	
[Response in frame: 352]	
[Next request in frame: 375]	

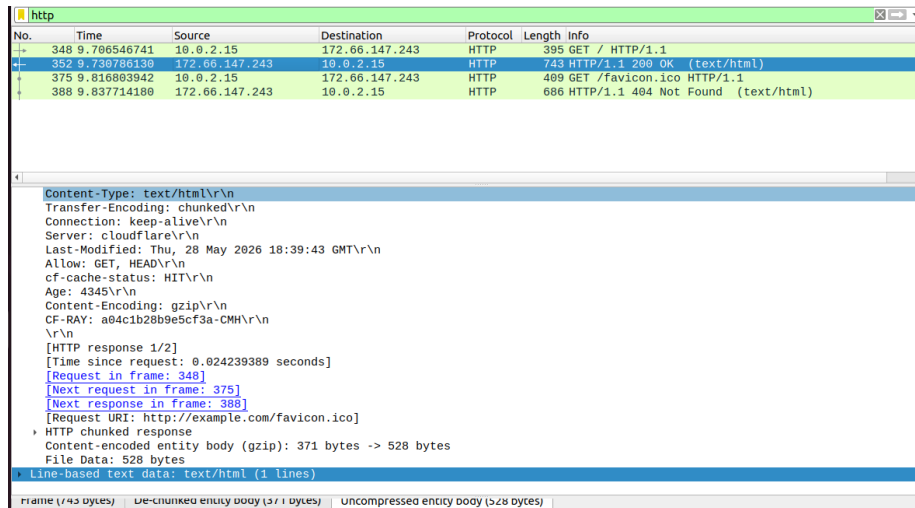
Figure 2: HTTP Request to “<http://example.com>”

Task 2: Understanding HTTP using telnet and Wireshark

We used the terminal-based telnet utility to connect to an HTTP server on port 80 and manually issue a minimal GET request, allowing us to view the raw server response directly in the console. Simultaneously, we captured this exchange in Wireshark to inspect the packet data

1.

2. **Ans.** Yes, there is a massive difference between the manual Telnet request and the browser request. Our manual Telnet request was extremely minimal, containing only the request line (GET / HTTP/1.0) and a single header (Host: example.com).



No.	Time	Source	Destination	Protocol	Length	Info
348	9.706546741	10.0.2.15	172.66.147.243	HTTP	395	GET / HTTP/1.1
352	9.730786130	172.66.147.243	10.0.2.15	HTTP	743	HTTP/1.1 200 OK (text/html)
375	9.816803942	10.0.2.15	172.66.147.243	HTTP	409	GET /favicon.ico HTTP/1.1
388	9.837714180	172.66.147.243	10.0.2.15	HTTP	686	HTTP/1.1 404 Not Found (text/html)


```

Content-Type: text/html\r\n
Transfer-Encoding: chunked\r\n
Connection: keep-alive\r\n
Server: cloudflare\r\n
Last-Modified: Thu, 28 May 2026 18:39:43 GMT\r\n
Allow: GET, HEAD\r\n
cf-cache-status: HIT\r\n
Age: 4345\r\n
Content-Encoding: gzip\r\n
CF-RAY: a04c1b28b9e5cf3a-CMH\r\n
\r\n
[HTTP response 1/2]
[Time since request: 0.024239389 seconds]
[Request in frame: 348]
[Next request in frame: 375]
[Next response in frame: 388]
[Request URI: http://example.com/favicon.ico]
  HTTP chunked response
  Content-encoded entity body (gzip): 371 bytes -> 528 bytes
  File Data: 528 bytes
  Line-based text data: text/html (1 lines)
  frame (743 bytes)  DE-chunked entity body (371 bytes)  Uncompressed entity body (528 bytes)

```

Figure 3: HTTP Response from “http://example.com”

3. Ans. Yes, The browser response optimizes the exchange by keeping the connection open (keep-alive) and compressing the data (gzip). In contrast, the manual Telnet response receives raw, uncompressed text and immediately closes the connection (close).

Part 2: Basic Web Application Programming

Task 1: CGI Web applications in C

a: We wrote a basic C program that outputs standard HTTP text headers, compiled it using gcc, and moved the executable into the server’s CGI directory. After ensuring the file had the correct execution permissions, we successfully invoked the script through our web browser to display the dynamic output.



Figure 4: HTTP Stream for “http://example.com”

```

GET /index.html HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:151.0)
Gecko/20100101 Firefox/151.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Priority: u=0, i

HTTP/1.1 404 Not Found
Date: Sun, 31 May 2026 22:41:43 GMT
Content-Type: text/html
Transfer-Encoding: chunked
Connection: keep-alive
Server: cloudflare
Age: 5198
cf-cache-status: HIT
Content-Encoding: gzip
CF-RAY: a0497cd8aab4cf3a-CMH

<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
GET /favicon.ico HTTP/1.1
Host: example.com

```

Figure 5: HTTP Stream for “http://example.com/index.html” which shows 404 Not Found

```

vboxuser@SpecialStudies:~$ telnet example.com 80
Trying 104.20.23.154...
Connected to example.com.
Escape character is '^]'.
GET / HTTP/1.0
Host: example.com

HTTP/1.1 200 OK
Date: Sun, 31 May 2026 23:08:13 GMT
Content-Type: text/html
Connection: close
Server: cloudflare
Last-Modified: Thu, 28 May 2026 04:54:11 GMT
Allow: GET, HEAD
Accept-Ranges: bytes
Age: 7279
cf-cache-status: HIT
CF-RAY: a049a3a4be8f24d3-CMH

<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
Connection closed by foreign host.
vboxuser@SpecialStudies:~$

```

Figure 6: terminal showing the HTTP Request and HTTP response from the server.

No.	Time	Source	Destination	Protocol	Length	Info
13	11.493880763	10.0.2.15	172.66.147.243	HTTP	58	GET / HTTP/1.0
17	11.515273713	172.66.147.243	10.0.2.15	HTTP	62	HTTP/1.1 200 OK (text/html)

Frame 13: 58 bytes on wire (464 bits), 58 bytes captured (464 bits) on interface any, id 0 Linux cooked capture v1 Internet Protocol Version 4, Src: 10.0.2.15, Dst: 172.66.147.243 Transmission Control Protocol, Src Port: 33336, Dst Port: 80, Seq: 36, Ack: 1, Len: 2 [3 Reassembled TCP Segments (37 bytes): #9(16), #11(19), #13(2)]	
Hypertext Transfer Protocol GET / HTTP/1.0\r\n Host: example.com\r\n \r\n [Full request URI: http://example.com/] [HTTP request 1/1] [Response in frame: 17]	

Figure 7: HTTP Request message in Wireshark

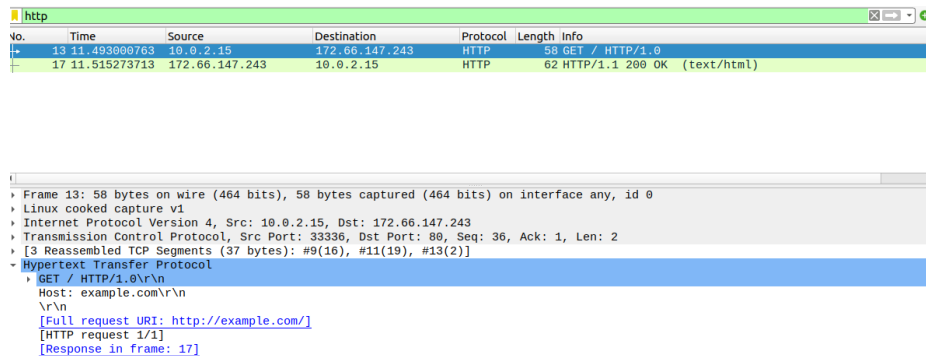
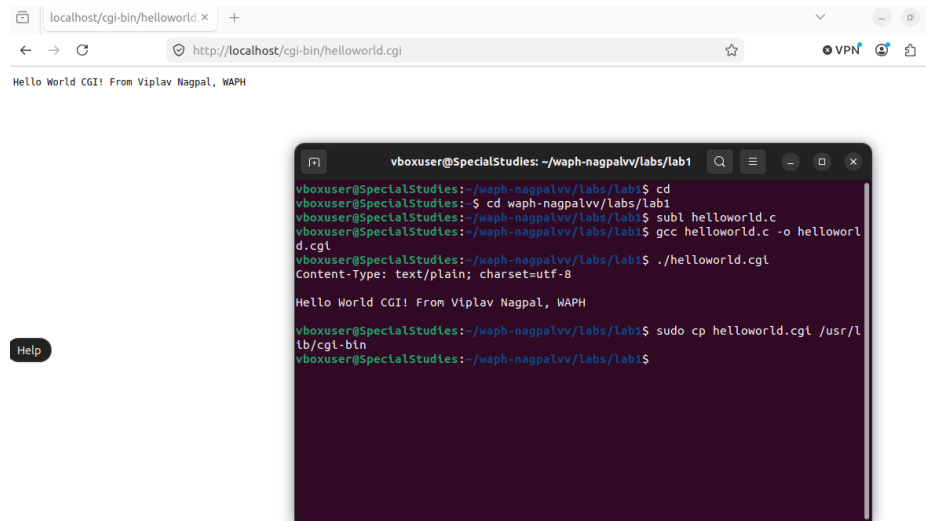


Figure 8: HTTP Response message in Wireshark that the server responded



b:

Task 2: A simple PHP Web Application with user input

a: We authored a helloworld.php script containing an echo statement for our name and the phpinfo() function to expose the server's environment settings. We placed this file in the web root directory and navigated to it in our browser, verifying the server successfully interpreted and executed the PHP code.

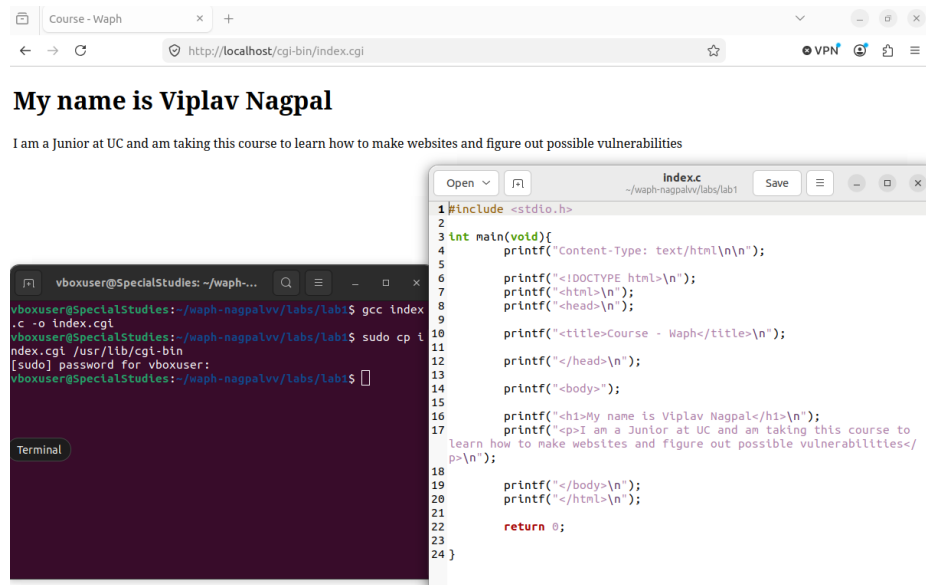
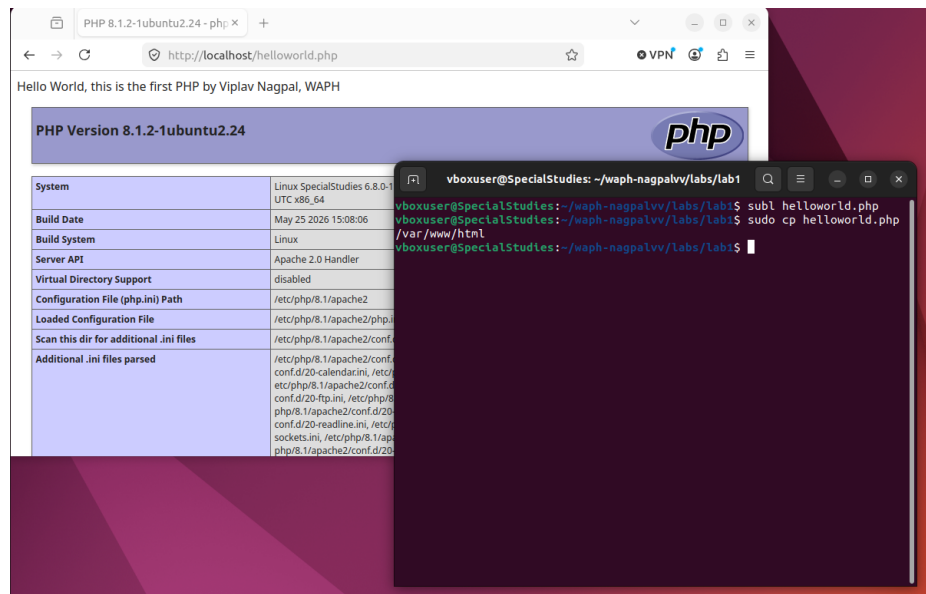
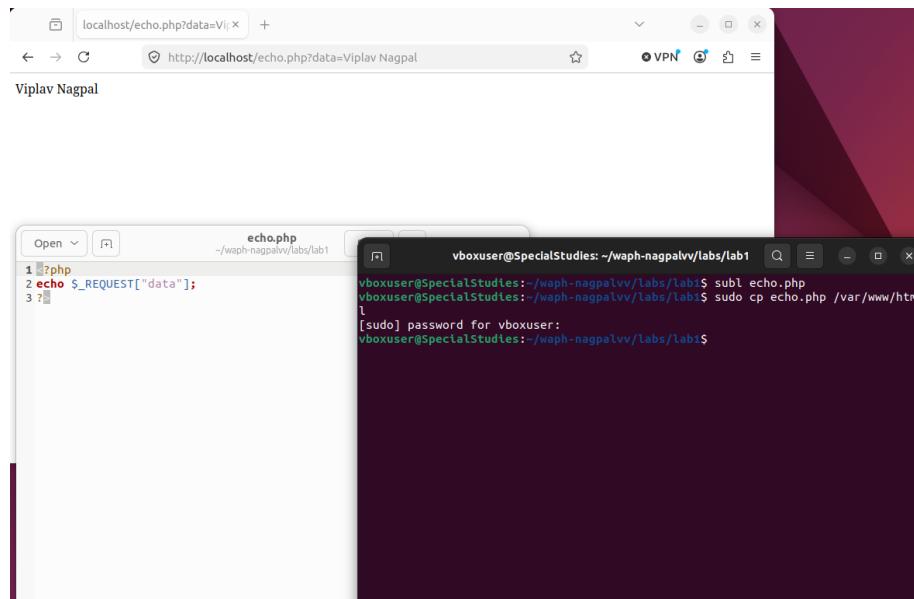


Figure 9: Source Code of the C code I wrote on my own and the resulting webpage





b:

This creates a critical Cross-Site Scripting (XSS) vulnerability, as the application fails to sanitize input, allowing malicious executable scripts to be injected and run.

Task 3: Understanding HTTP GET and POST requests

a: We started a Wireshark packet capture, accessed the echo.php page with our name appended as a URL parameter, and filtered the captured traffic for HTTP. Inspecting the captured packets revealed our specific input explicitly visible within the requested URI string of the HTTP GET Request message.



b: We executed the curl command with the -d flag to manually transmit an HTTP POST request containing our name as payload data to the server. Running Wireshark concurrently allowed us to capture this traffic and verify that the POST request successfully delivered the data hidden within the message body.

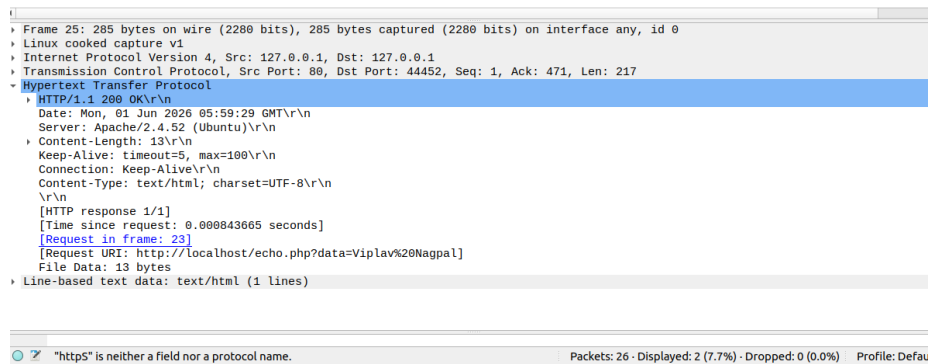
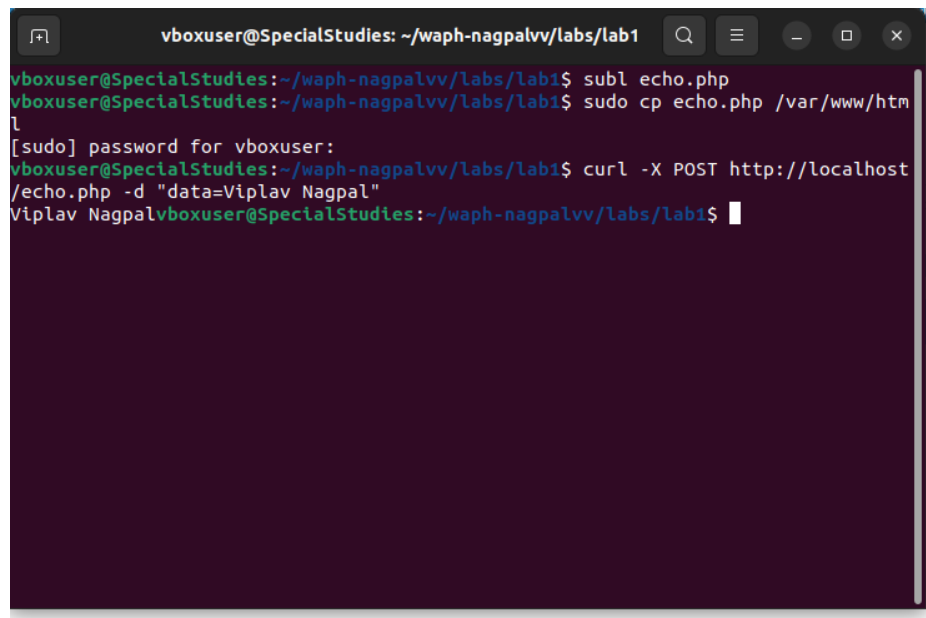


Figure 10: HTTP GET Response for the echo.php page with my name in the data



No.	Time	Source	Destination	Protocol	Length	Info
4	0.000127430	127.0.0.1	127.0.0.1	HTTP	237	POST /echo.php HTTP/1.1 (application/x-www-form-urlencoded)
6	0.00020842	127.0.0.1	127.0.0.1	HTTP	229	HTTP/1.1 200 OK (text/html)

Frame 4: 237 bytes on wire (1896 bits), 237 bytes captured (1896 bits) on interface any, id 0

- Linux cooked capture v1
- Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
- Transmission Control Protocol, Src Port: 41672, Dst Port: 80, Seq: 1, Ack: 1, Len: 169
- Hypertext Transfer Protocol
 - POST /echo.php HTTP/1.1\r\n
 - Host: localhost\r\n
 - User-Agent: curl/7.81.0\r\n
 - Accept: */*\r\n
 - Content-Length: 18\r\n
 - Content-Type: application/x-www-form-urlencoded\r\n
 - \r\n
 - [Full request URI: http://localhost/echo.php]
 - [HTTP request 1/1]
 - [Response in frame: 6]
 - File Data: 18 bytes
- HTML Form URL Encoded: application/x-www-form-urlencoded

No.	Time	Source	Destination	Protocol	Length	Info
4	0.000127430	127.0.0.1	127.0.0.1	HTTP	237	POST /echo.php HTTP/1.1 (application/x-www-form-urlencoded)
6	0.00020842	127.0.0.1	127.0.0.1	HTTP	229	HTTP/1.1 200 OK (text/html)

Frame 6: 229 bytes on wire (1832 bits), 229 bytes captured (1832 bits) on interface any, id 0

- Linux cooked capture v1
- Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
- Transmission Control Protocol, Src Port: 80, Dst Port: 41672, Seq: 1, Ack: 170, Len: 161
- Hypertext Transfer Protocol
 - HTTP/1.1 200 OK\r\n
 - Date: Mon, 01 Jun 2026 07:41:44 GMT\r\n
 - Server: Apache/2.4.52 (Ubuntu)\r\n
 - Content-Length: 13\r\n
 - Content-Type: text/html; charset=UTF-8\r\n
 - \r\n
 - [HTTP response 1/1]
 - [Time since request: 0.000893412 seconds]
 - [Request in frame: 4]
 - [Request URI: http://localhost/echo.php]
 - File Data: 13 bytes
- Line-based text data: text/html (1 lines)

Figure 11: HTTP POST Response for the echo.php page with my name in the data

c: While both methods yielded identical 200 OK server responses and HTML output, they differed completely in how they transmitted the user data to the server. The GET request exposed our input openly within the URL string, whereas the POST request safely encapsulated the input within the HTTP message body.

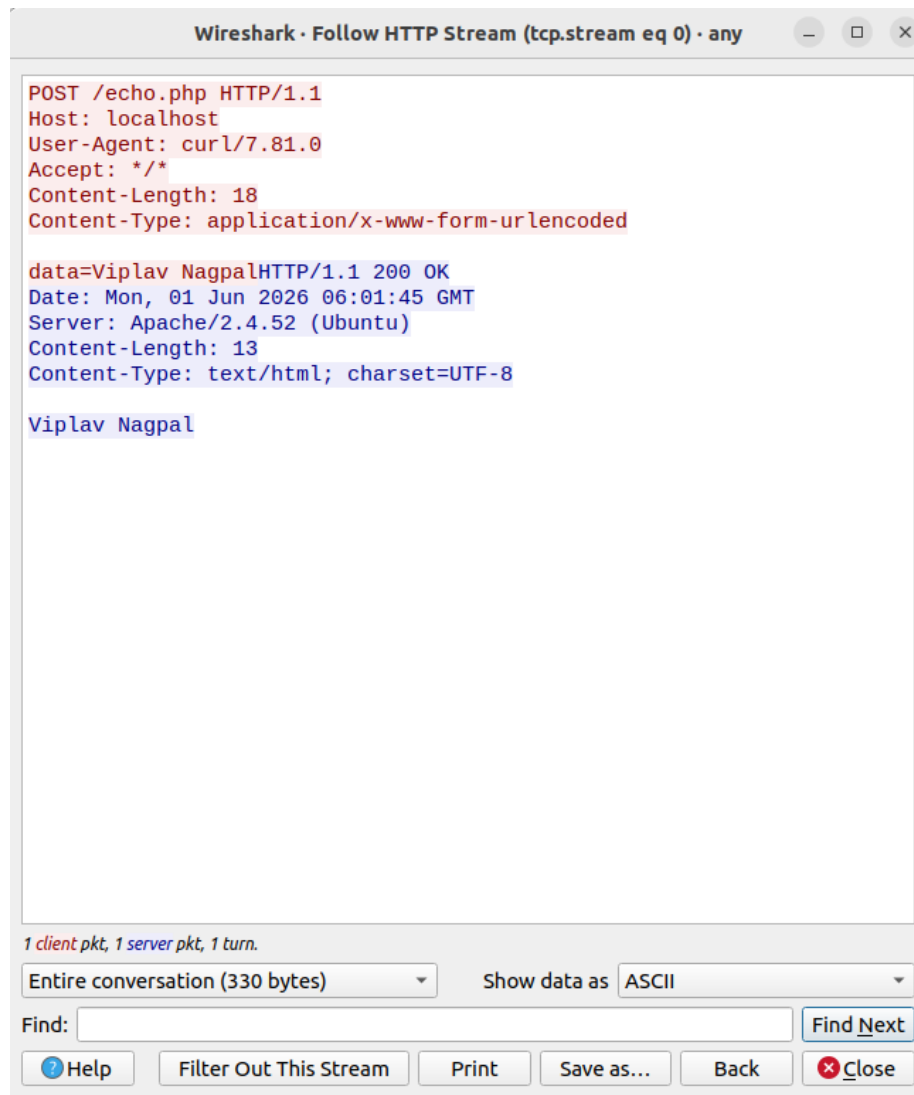


Figure 12: HTTP stream for echo.php through POST